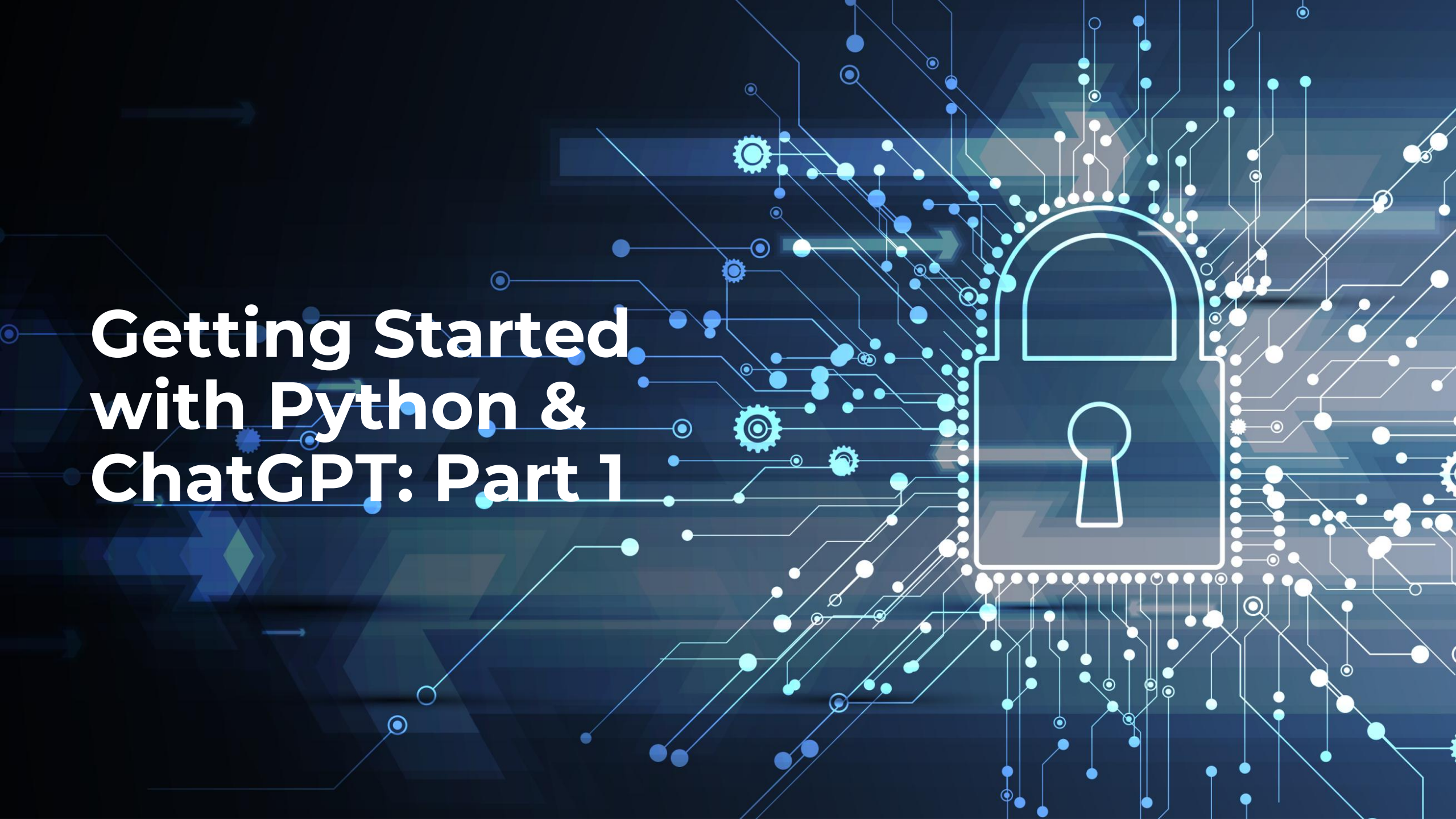
IITM PRAVARTAK
CATALYSING INNOVATION

Professional Certificate Programme in Agentic AI and Applications

Getting Started with Python & ChatGPT: Part 1



Topics Covered

- The Journey of AI
- Agentic AI
- Limitations and Advantages of Current AI
- Real-World Use Cases of Agentic AI
- Key Components of Agentic AI
- Getting Started with ChatGPT and Python
- Writing Python Scripts

The Journey of AI

The Journey of AI

- Evolves from a single technology into a broad spectrum of capabilities
- Advances through stages from **pattern recognition** to **content generation** to **autonomous action**
- Reveals new opportunities and limitations at each stage

Traditional AI v. Generative AI v. Agentic AI

Aspect	Traditional AI	Generative AI	Agentic AI
Core Capability	Pattern recognition, predictions	Content creation (text, images, code)	Autonomous reasoning, planning & acting
Input–Output	Historical data → classification	Prompt → generated output	Goal/task → multi-step execution
Human Involvement	High; relies on rules and reprogramming	Medium; guided by prompts	Low; makes autonomous decisions
Examples	Fraud detection, credit scoring, recommendation engines	ChatGPT, Stable Diffusion, GitHub Copilot	End-to-end travel booking, automated compliance, workflow orchestration
Limitations	Narrow and rule based; not adaptive	Lacks initiative, needs human integration	Early stage; control and safety are complex

Evolution of Traditional AI to Agentic AI

Represents how Machines Interact and Impact the World

Focusses on pattern recognition, classification and predictions based on historical data

- Fraud detection systems
- Recommendation engines
- Statistical analysis tools

Traditional AI

Creates content in text, images audio and code from existing patterns

- Chatgpt conversations
- Stable diffusion images
- Code completion tools

Generative AI

Reasons and plans independently and acts across multiple tools and systems

- End-to-end travel booking
- Financial record reconciliation
- Complete workflow management

Agentic AI

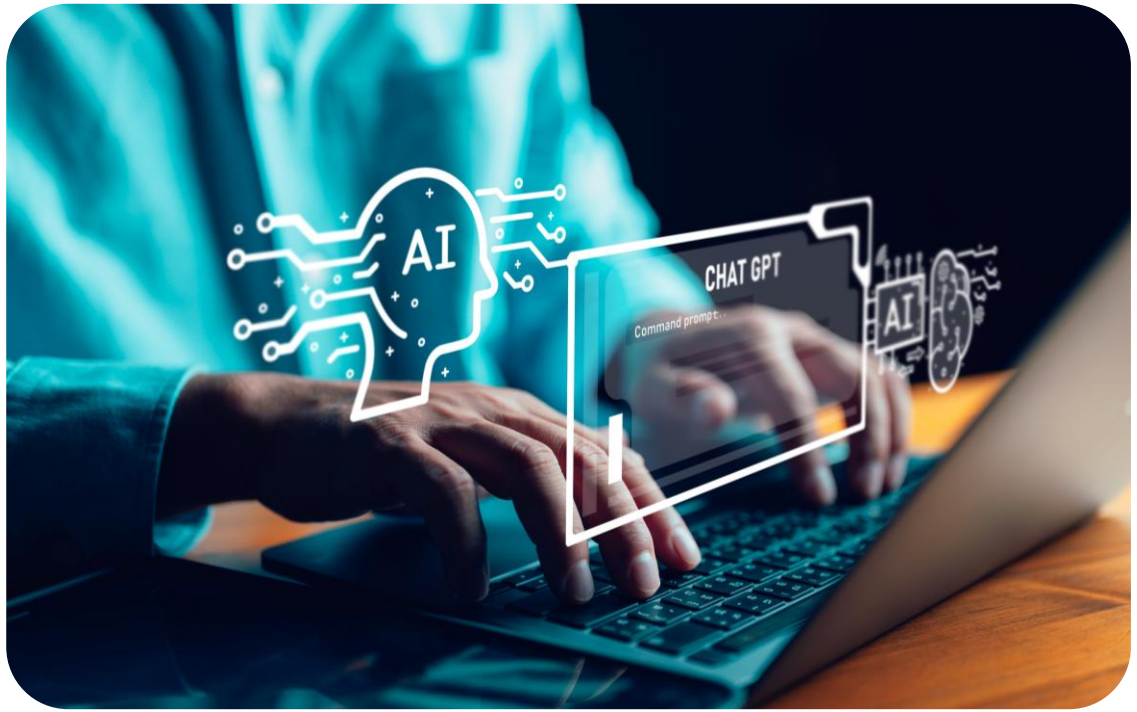
Applications Across AI Types

Traditional AI

- **Focus:** Predictions & pattern recognition
- **Applications**: Fraud detection, credit scoring, recommendation engines
- **Outcome:** Deliver efficiency in narrow, rule-based tasks

Applications Across AI Types

Generative AI



Focus: Content creation

Applications: Conversational assistants, image and video generation, code completion

Outcome: Expand human creativity across text, images and code

Applications Across AI Types

Agentic AI

Focus: Perform reasoning, planning and autonomous action

Applications: End-to-end travel booking, automate compliance and workflow orchestration

Outcome: Act as an autonomous partner in complex workflows

Agentic AI

Significance of Agentic AI

Beyond predictions & content

Traditional AI predicts, Generative AI creates, but both rely on human input and oversight

Need for autonomy

Modern demands require AI that reasons, plans and acts independently

Bridging gaps

Agentic AI links output to real-world actions, enabling execution

Significance of Agentic AI

Business Value

- Automates multi-step workflows
- Enhances decision-making with context
- Scales operations without proportional staffing
- Frees humans for creative & strategic work

Agentic AI

Agentic AI refers to systems that function as **autonomous agents**, capable of reasoning, planning and executing complex, multi-step tasks with minimal human guidance.

Agentic AI

Autonomy

Acts independently without constant human prompting, making decisions based on available information

Goal-oriented

Works toward objectives rather than just answering queries, planning steps to achieve desired outcomes

Multi-tool use

Combines APIs, data sources and applications dynamically to accomplish complex tasks

Adaptive

Learns continuously from feedback, context and environmental changes to improve performance

Limitations and Advantages of Current AI

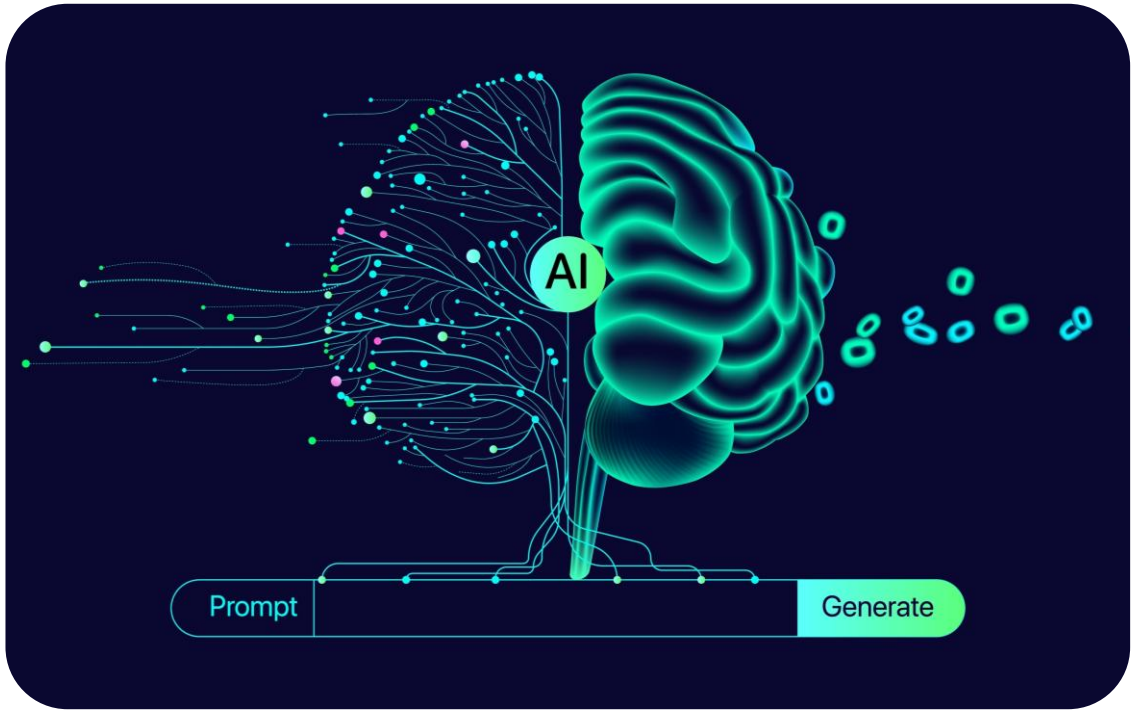
Limitations of Current AI

Traditional AI

- Operates within fixed rules and training data
- Fails to adapt to new tasks without human reprogramming
- Excels at prediction but struggles with decision-making
- Handles narrow, repetitive problems effectively like credit scoring, spam detection

Limitations of Current AI

Generative AI



- Produces human-like text, image or code but stops at content creation
- Relies heavily on prompts, lacks initiative to pursue multi-step goals
- Lacks built-in mechanisms to validate outputs against real-world outcomes
- Depends on humans to integrate outputs into workflows

Advantages of Agentic AI

Autonomy
Initiates and completes tasks without explicit step-by-step instructions

Reasoning
Evaluates options, applies logic and adapts strategies

Action
Interacts with external systems like APIs, tools, databases to execute real-world tasks

Continuous learning
Improves from feedback, making processes more efficient over time

Human + AI collaboration
Frees humans from repetitive tasks so they focus on strategic, creative work

Business Relevance of Agentic AI

Delivers Transformative Capabilities Across the Enterprise

Efficiency	Automates complex, multi-step workflows, speeding processes
Decision support	Adds reasoning and context for better strategic choices
Scalability	Manages complex processes with minimal oversight
Integration	Connects across APIs, databases and systems to break silos
Competitive edge	Transforms experiences and operations for market differentiation

Real-World Use Cases of Agentic AI

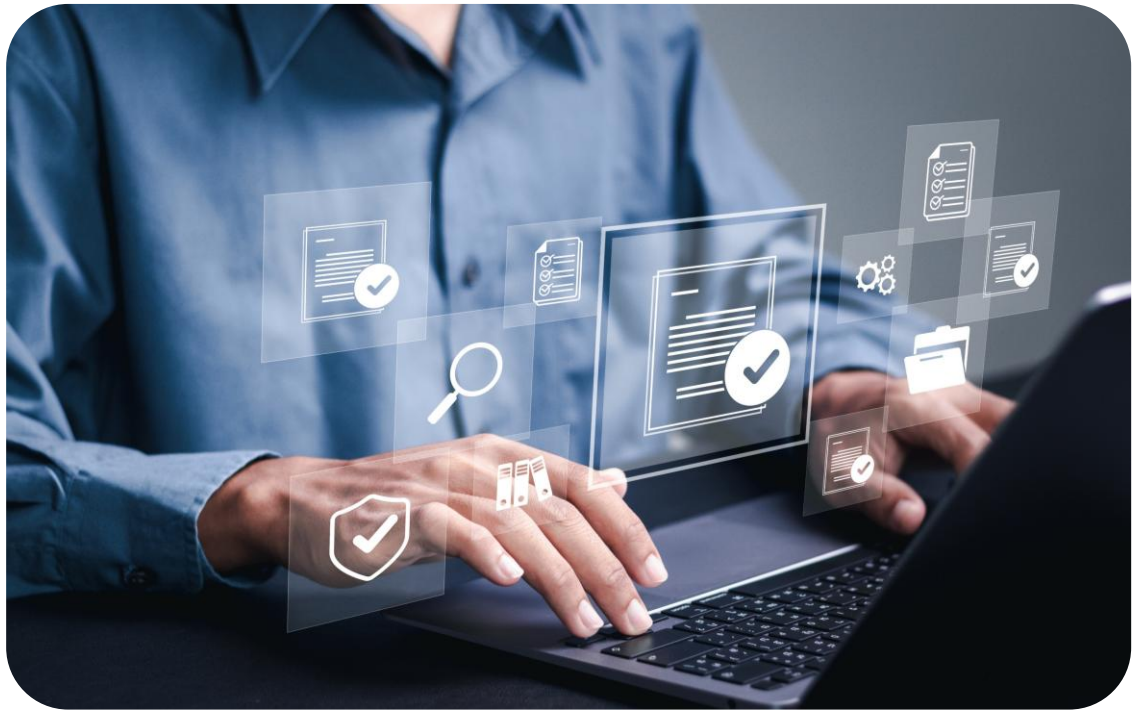
Real-World Use Cases of Agentic AI

Finance

- Automates financial analysis across data sources
- Conducts risk assessment using diverse inputs
- Generates compliance reports aligned with regulations

Real-World Use Cases of Agentic AI

Customer Support



- Resolves issues end-to-end with AI agents
- Accesses knowledge bases, account data and service tools
- Eliminates human handoffs in customer support

Real-World Use Cases of Agentic AI

Procurement

- Reviews contracts
- Manages vendors efficiently
- Optimises purchases based on needs and market conditions

Real-World Use Cases of Agentic AI

Business Development

- Qualifies leads end-to-end
- Generates proposals automatically
- Schedules seamlessly with CRM and calendars

Real-World Use Cases of Agentic AI

Healthcare

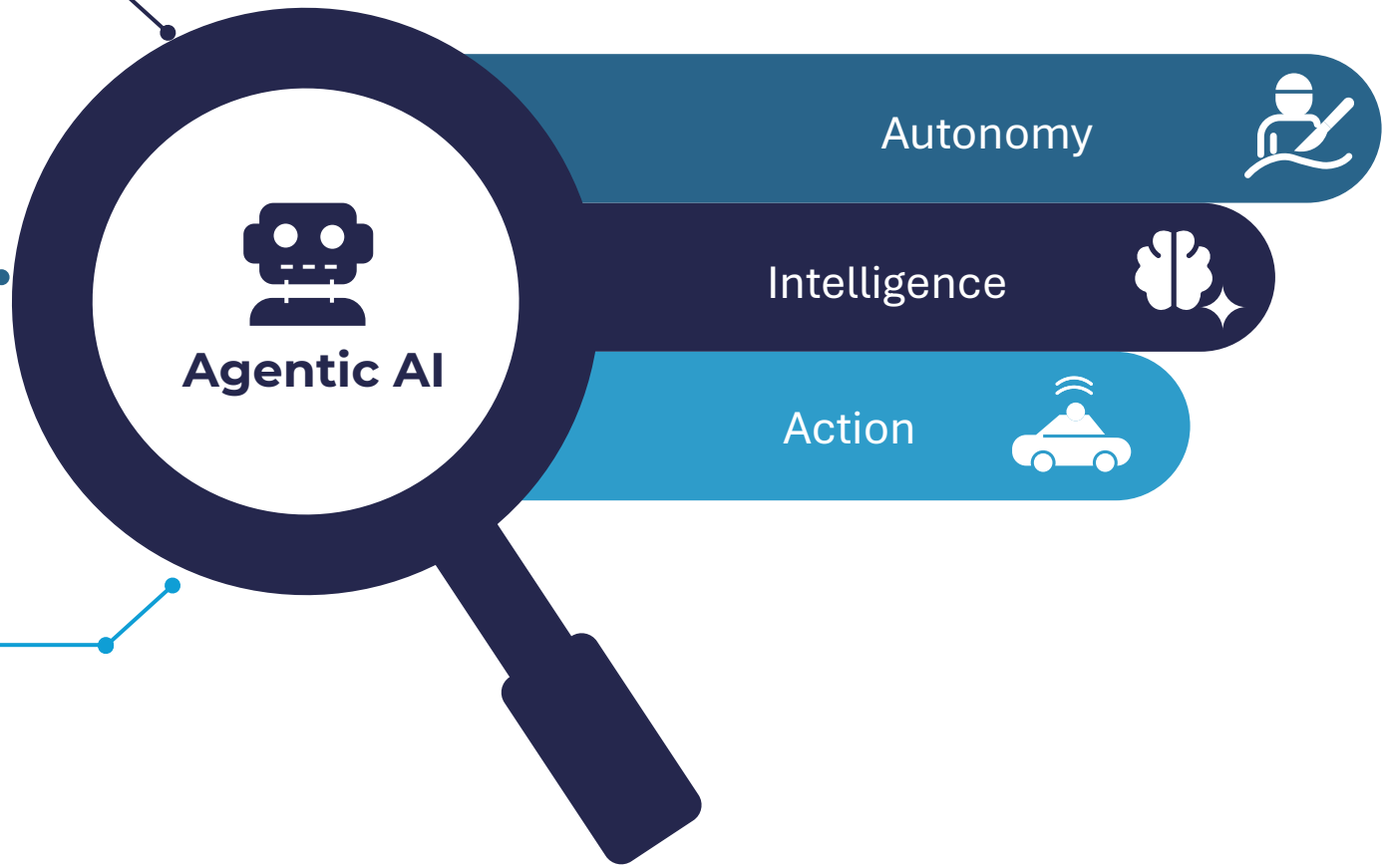
- Triages patients efficiently
- Coordinates with medical systems
- Schedules and manages follow-ups

Agentic AI

Acts independently without continuous human guidance

Makes informed decisions through reasoning and learning

Executes tasks across systems to achieve goals



Agentic AI shifts from assistive tools to autonomous partners, redefining business operations and innovation.

Key Components of Agentic AI

Key Components of Agentic AI

Large Language Models (LLMs)

- Understand and generate human-like text
- Enable reasoning, summarization, and knowledge retrieva
- Examples: GPT, LLaMA, Claude, Gemini

Tool use and APIs

- Connect to external systems (databases, apps, APIs)
- Execute actions beyond text creation

Reasoning and planning

- Break down complex goals into logical steps
- Adapt strategies based on feedback and changing context

Autonomy and action

- Operate with minimal human prompting
- Complete multi-step workflows end-to-end

Getting Started with Python & ChatGPT

Introduction to ChatGPT

- Developed by OpenAI as a Large Language Model
- Designed to understand and generate human-like text

Key Strengths of ChatGPT

Engages in conversational interaction

Generates and explains code

Retrieves knowledge and provides summaries

Assists across domains like education, business and software development

Applications Across Domains

Education

- Provides learning support and personalised tutoring
- Generates content for lessons and materials
- Answers student questions in real time

Applications Across Domains

Business

- Drafts reports and business communications
- Automates customer support
- Conducts market research and data analysis

Applications Across Domains

Software Development

- Debugs complex code issues
- Learns new programming concepts
- Generates efficient code solutions

Applications Across Domains

Creative Work

- Creates storytelling and content ideas
- Develops design concepts and brainstorm
- Refines and edits creative projects

Why Developers Use ChatGPT

Debugging

Identifies and fixes errors quickly

Coding assistance

Automates routine coding tasks

Learning

Explores new frameworks, languages and concepts

Collaboration

Acts as a 24/7 coding partner

Debugging with ChatGPT

Provides clear explanations of complex error messages, identifying the issue within seconds

Suggests likely causes of bugs by analysing code patterns and framework pitfalls

Offers multiple fixes so developers can choose the most suitable solution

Error interpretation

Root cause analysis

Solution alternatives

Example: Resolves a hidden Python dependency conflict in minutes

Learning with ChatGPT

- Explains concepts in simple, step-by-step terms
- Compares libraries and frameworks
- Suggests resources, examples and best practices

Learning with ChatGPT

Example

Prompt: "Explain difference between pandas loc and iloc" → clear explanation with code samples

```
# Example of pandas loc vs ilocdf.loc[1:3, 'Name'] # Label-
baseddf.iloc[1:3, 0] # Position-based
```

Coding Assistance

Boilerplate generation

Generates starter code for common patterns without recalling syntax details

Code refactoring

Converts legacy code into cleaner, maintainable versions following best practices

Optimisation

Spots performance bottlenecks and suggests faster, more efficient alternatives

Coding Assistance

Example

```
# Example: Ask ChatGPT to generate a Python functiondef
    fetch_weather_data(city, api_key): """Retrieve
current weather data for specified city."""  import requests base_url
    =
"https://api.weatherapi.com/v1/current.json" params = {"key": api_key
    , "q": city} response =
requests.get(base_url, params=params) if response.status_code == 200:
    return response.json() else: return
{"error": f"Failed with status code {response.status_code}"}
```

Real-World Case Studies

GitHub copilot integration

Developers achieve up to 55% faster coding, particularly for repetitive tasks and boilerplate code.

Stack overflow assistance

Developers use ChatGPT to ask better questions and interpret answers, speeding up problem-solving.

Development cycle reduction

Teams reduce iteration cycles by 30–40% in documentation, testing, and API integration with ChatGPT.

ChatGPT enhances developers' work, augmenting human capabilities without replacing them.

Benefits for Professionals

40% – Time saved

Reduce time spent on debugging and problem-solving with AI assistance

24/7 – Availability

Access learning resources and coding support anytime, anywhere

85% – Adoption rate

Most developers continue using AI tools after trial, showing strong value

3x – Learning speed

Learn new frameworks and languages up to three times faster with AI support

- Debugging
- Documentation
- Learning
- Code Generation
- Testing

ChatGPT boosts productivity, especially in debugging and documentation.

Limitations to Keep in Mind

Code quality concerns

May generate incorrect or outdated code

Human oversight required

Needs validation and testing by developers

Supplementary tool

Works best as a co-pilot rather than a replacement

Essential Skills for Agentic AI

Builds the foundation for coding, experimentation and agents

Python programming skills

Understanding of Large Language Models (LLMs)

Knows how GPT, LLaMA, Claude and Gemini process and generate text

Creates prompts that guide AI behaviour and outputs

Prompt engineering basics

Relevant domain knowledge

Applies Agentic AI in business, education, healthcare and beyond

Why Python for AI & Agentic AI?

Simplicity & readability

- Easy-to-learn syntax
- Used in academia and industry

Rich ecosystem

- Libraries like TensorFlow, PyTorch, LangChain, OpenAI SDK
- Tools for data, visualisation and automation

Versatility

- Used in web, data science, automation and AI
- Supports rapid prototyping and experimentation

Community support

- Large global community with tutorials and open-source packages
- Faster troubleshooting and learning

Introduction to Python

- Is a high-level, interpreted programming language
- Offers simplicity, readability and versatility

Python for Agentic AI

Key Advantages

- Offers simplicity with readable syntax and low entry barriers
- Provides a rich ecosystem of AI/ML libraries (TensorFlow, PyTorch, OpenAI SDK)
- Delivers extensive documentation and community support
- Enables rapid prototyping capabilities

Python for Agentic AI

Agentic AI Capabilities

Makes complex decisions using multiple data sources

Manage workflows across various tools and services

Adapts to changing environments and requirements

Collaborates with humans and other AI agents

Python-powered agentic AI is transforming automation and decision-making, from trading systems to intelligent assistants.

Python in Action: Frameworks & Practical Usage

Leading Python Frameworks

LangChain

Enables chain-of-thought reasoning and tool integration

CrewAI

Supports multi-agent collaboration and role-based teams

Microsoft Autogen

Facilitates conversational agent orchestration

OpenAI Agents SDK

Enables function calling and tool usage

Python in Action: Frameworks & Practical Usage

Real-World Applications

Autonomous teams for debugging and improving code

Financial agents for processing market data in real-time

Content systems for researching, writing and illustrating reports

Customer service agents for handling complex inquiries

Python in Action: Frameworks & Practical Usage

Building Agentic AI with Python

Start small: Focus on Python projects solving specific problems with agentic capabilities

Integrate tools: Connect agents to APIs, databases and services to enhance capabilities

Scale gradually: Build complex multi-agent systems as you gain expertise

Python and agentic AI frameworks enable developers to drive automation and manage complexity incrementally.

Why Use Python with Jupyter or Colab?

Python: The Foundation

- Simplifies data science, AI and automation tasks
- Offers a vast ecosystem of libraries for experts and beginners

Why Use Python with Jupyter or Colab?

Jupyter Notebooks: Interactive Computing

- Combines code, text, Maths, plots and widgets in one document
- Supports exploration, teaching and collaboration

Google Colab: Power in the Cloud

- Grants free access to GPU/TPU resources
- Enables real-time sharing and collaboration on notebooks

Step 1: Installing Python on Your Computer

Download: Download the latest stable version (3.11+ recommended) from python.org

Installation options: Check "Add Python to PATH" for command-line access

Verification: Run python --version in the terminal to confirm installation

Step 2: Installing Jupyter Notebook Locally

- Open your terminal or command prompt
- Run the pip install command:

```
pip install notebook
```

- Use the command to launch Jupyter:

```
jupyter notebook
```

- Your default browser will open to the dashboard at http://localhost:8888

Click "New → Python 3" from the dashboard to create your first notebook

Step 3: Installing JupyterLab (Optional Advanced Interface)

- Install JupyterLab using the pip package manager:

```
pip install jupyterlab
```

- Launch JupyterLab with the following command:

```
jupyter lab
```

- A browser window will open automatically at http://localhost:8888/lab

Step 3: Installing JupyterLab

Key Advantages

- Open multiple tabs for notebooks, terminals and text editors in one window
- Use an integrated file browser with drag-and-drop functionality
- Customise the layout with resizable panels
- Access an extensive plugin ecosystem for added functionality
- Experience improved rendering of data visualisations

Step 4: Using Google Colab Without Installation

Access Colab

Visit colab.research.google.com and sign in with your Google account

Create or open notebooks

Click "New notebook" or open existing notebooks from Google Drive, GitHub or local upload

Access free computing resources

Select Runtime → Change runtime type to use free GPU or TPU acceleration

Colab sessions disconnect after 90 minutes of inactivity or 12 hours, so save your work regularly.

Open a Downloaded Jupyter Notebook in Colab

Steps Involved

Upload Notebook
Drag and drop your .ipynb file into Colab or use File → Upload notebook

Use Google Drive
Upload to Google Drive first, then right-click → Open with → Google Colaboratory

Run and share
Execute cells with Shift+Enter or the play button and share via the Share button

Reinstall any required libraries in Colab with `!pip install package-name` at the session start.

Writing Python Scripts

Importance of Jupyter Notebook

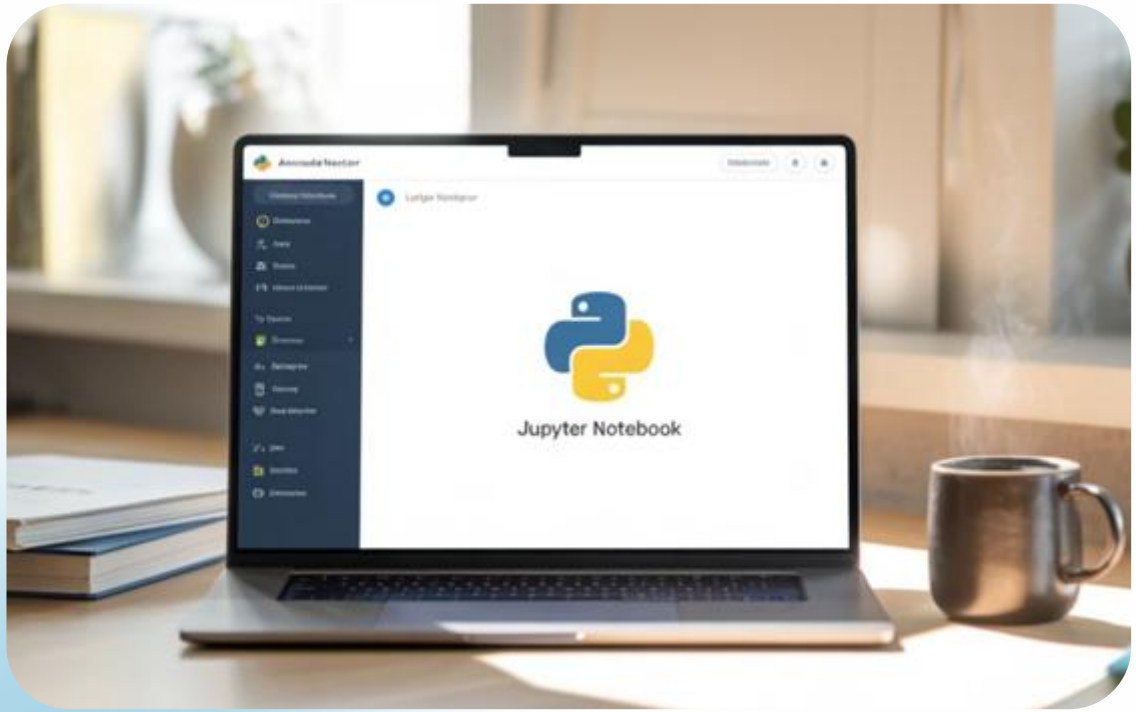
Provides an interactive environment for writing and running code in cells

Supports rich outputs such as tables, plots, text and images

Enables easy testing of code snippets step by step for beginners

Serves as a widely used tool in data science, ML and education

Getting Started with Jupyter



- Launch Jupyter Notebook using `jupyter notebook` or from Anaconda Navigator
- Create a new notebook (`.ipynb file`)
- Write code in cells and run with Shift + Enter

First Python Script in Jupyter

```
print("Hello, Python World!")
```

- Enter the code in a cell and press Shift + Enter
- View the output directly below the cell
- Receive interactive and immediate feedback

Expanding the Script

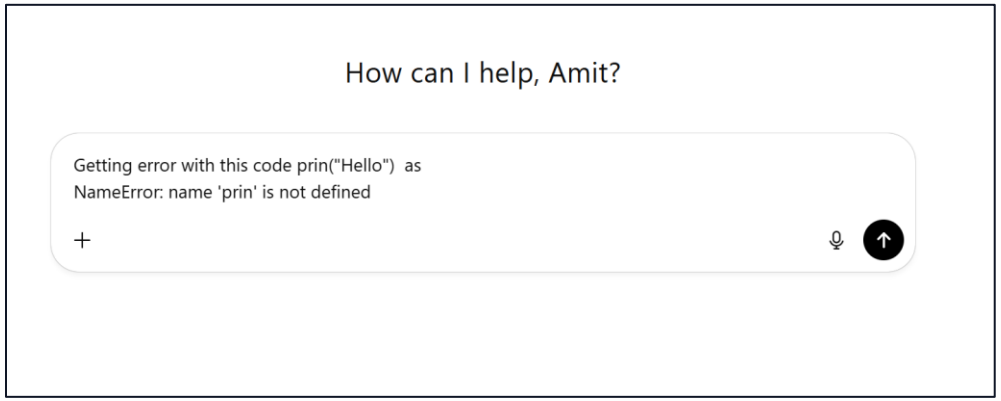
- Define variables a and b and print the sum

```
a = 5

b = 3

print("Sum is:", a + b)
```

- Introduce variables, operators, and print statements

- Allow learners to change values in real-time and rerun cells instantly

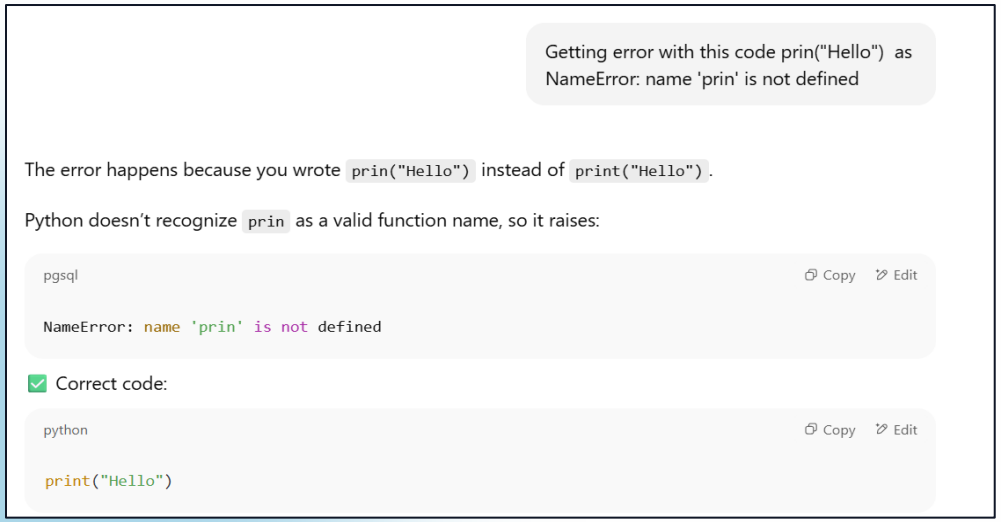
Debugging in Jupyter with ChatGPT



- Introduce an error in the notebook as

```
prin("Hello")
```

- Output: `NameError: name 'prin' is not defined`



- Copy the error message and code into ChatGPT
- Receive an explanation and correction as

```
print("Hello")
```

Significance of Jupyter + ChatGPT in Function

Show immediate error tracebacks in Jupyter

Explain errors in plain English with ChatGPT

Fix and rerun for an instant feedback cycle

Promote learning by doing

Evaluating ChatGPT's Debugging Help

Checklist

- Check if the corrected code runs in Jupyter
- Check for hallucinated functions or libraries
- Check if the explanation matches the error message
- Check if ChatGPT suggests clear, beginner-friendly fixes

Case Example

- Fix this error:

```
prin("Hello")
```

- ChatGPT Output: Suggests correction:

```
print("Hello")
```

Case Example

- Prompt: Explain why my loop doesn't run

```
for i in range(5)

print(i)
```

- ChatGPT Output: Suggests adding a missing colon:

```
for i in range(5):
```

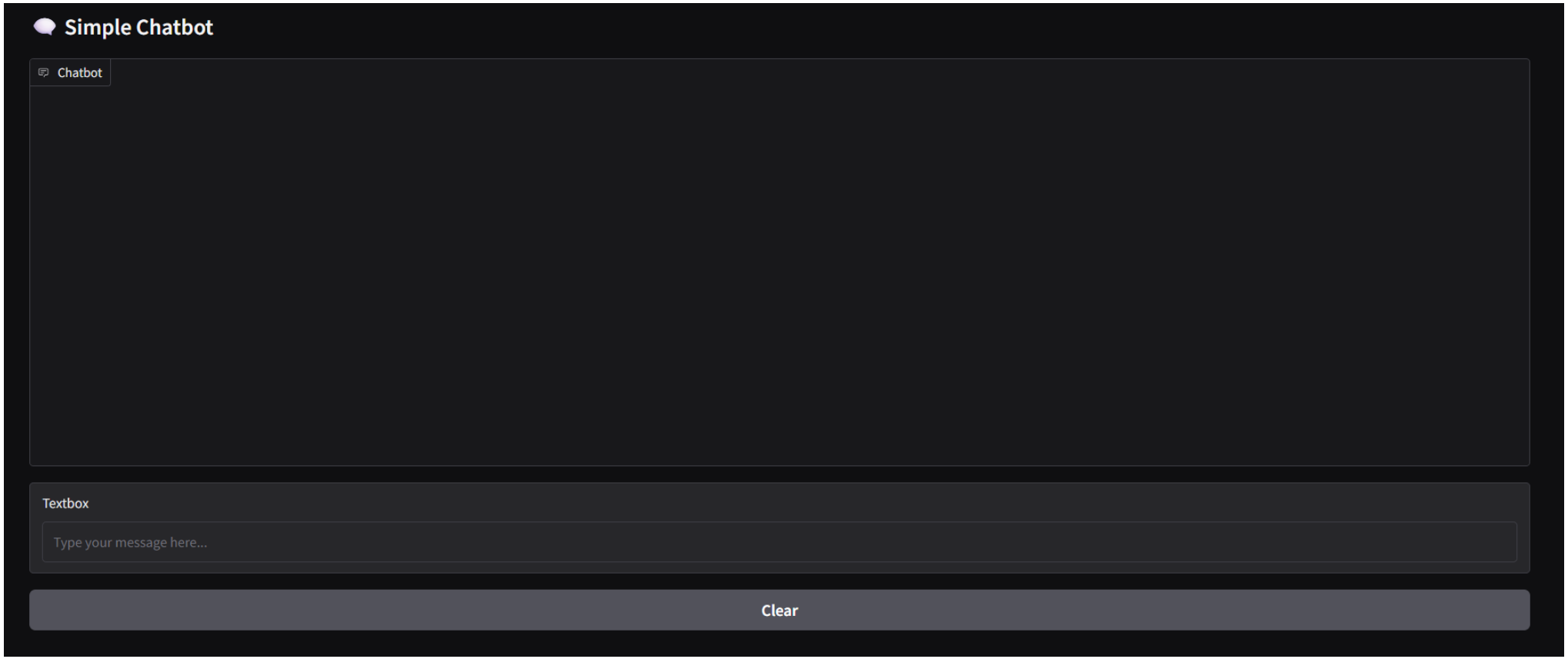
Generating Script with ChatGPT

- Prompt: Generate a code to create a chat bot interface with Gradio
- Output:

```
    import gradio as gr

#Define a simple chatbot function
def chatbot (user_input, history=[]):
#Basic echo bot logic (replace with your model logic)
response = "You said: (user_input}"

#Append to chat history
history.append((user_input, response))
return history, history
```

```
#Create the chatbot interface
with gr.Blocks() as demo:
gr.Markdown("# Simple Chatbot with Gradio")

chatbot_ui gr.Chatbot()
msg gr. Textbox(placeholder="Type your message here...)
clear gr.Button("Clear Chat")

#Events
msg.submit(chatbot, [msg, chatbot_ui], [chatbot_ui, chatbot_ui])
clear.click(lambda: None, None, chatbot_ui, queue=False)

#Launch the app
demo.launch()
```

Generating Script with ChatGPT



Q & A

Thank you